JavaScript is disabled on your browser.



compact1, compact2, compact3

java.util

# Interface Map.Entry<K,V>

**All Known Implementing Classes:**

AbstractMap.SimpleEntry, AbstractMap.SimpleImmutableEntry

**Enclosing interface:**

Map<K,V>

---

public static interface **Map.Entry<K,V>**

A map entry (key-value pair). The Map.entrySet method returns a collection-view of the map, whose elements are of this class. The *only* way to obtain a reference to a map entry is from the iterator of this collection-view. These Map.Entry objects are valid *only* for the duration of the iteration; more formally, the behavior of a map entry is undefined if the backing map has been modified after the entry was returned by the iterator, except through the setValue operation on the map entry.

**Since:**

1.2

**See Also:**

Map.entrySet()

## *Method Summary*

**Default Methods** **Abstract M**

| Modifier and Type | Method | Descrip |
|---|---|---|
| static <K extends Comparable<? super K>,V> Comparator<Map.Entry<K,V>> | comparingByKey() | Returns compar that compar |

| | | |
|---|---|---|
| | | Map.En natural on key. |
| static <K,V> Comparator<Map.Entry<K,V>> | comparingByKey(Comparator<? super K> cmp) | Returns compar that compar Map.En key usi given Compar |
| static <K,V extends Comparable<? super V>> Comparator<Map.Entry<K,V>> | comparingByValue() | Returns compar that compar Map.En natural on valu |
| static <K,V> Comparator<Map.Entry<K,V>> | comparingByValue(Comparator<? super V> cmp) | Returns compar that compar Map.En value u the give Compar |
| boolean | equals(Object o) | Compar specifie object v this ent equality |
| K | getKey() | Returns key corresp to this e |
| V | getValue() | Returns value corresp to this e |

| | | |
|---|---|---|
| int | **hashCode**() | Returns hash co value fo map en |
| **V** | **setValue**(**V** value) | Replace value corresp to this e with the specifie value (optiona operati |

## *Method Detail*

### getKey

K getKey()

Returns the key corresponding to this entry.

**Returns:**

the key corresponding to this entry

**Throws:**

IllegalStateException - implementations may, but are not required to, throw this exception if the entry has been removed from the backing map.

### getValue

V getValue()

Returns the value corresponding to this entry. If the mapping has been removed from the backing map (by the iterator's remove operation), the results of this call are undefined.

**Returns:**

the value corresponding to this entry

**Throws:**

IllegalStateException - implementations may, but are not required to, throw this exception if the entry has been removed from the backing map.

### setValue

`V setValue(V value)`

Replaces the value corresponding to this entry with the specified value (optional operation). (Writes through to the map.) The behavior of this call is undefined if the mapping has already been removed from the map (by the iterator's `remove` operation).

**Parameters:**

`value - new value to be stored in this entry`

**Returns:**

`old value corresponding to the entry`

**Throws:**

`UnsupportedOperationException - if the put operation is not supported by the backing map`

`ClassCastException - if the class of the specified value prevents it from being stored in the backing map`

`NullPointerException - if the backing map does not permit null values, and the specified value is null`

`IllegalArgumentException - if some property of this value prevents it from being stored in the backing map`

`IllegalStateException - implementations may, but are not required to, throw this exception if the entry has been removed from the backing map.`

### equals

`boolean equals(Object o)`

Compares the specified object with this entry for equality. Returns `true` if the given object is also a map entry and the two entries represent the same mapping. More formally, two entries `e1` and `e2` represent the same mapping if

```
(e1.getKey()==null ?
 e2.getKey()==null : e1.getKey().equals(e2.getKey()))  &&
(e1.getValue()==null ?
 e2.getValue()==null : e1.getValue().equals(e2.getValue()))
```

This ensures that the `equals` method works properly across different implementations of the `Map.Entry` interface.

**Overrides:**

equals in class Object

**Parameters:**

o - object to be compared for equality with this map entry

**Returns:**

true if the specified object is equal to this map entry

**See Also:**

Object.hashCode(), HashMap

### hashCode

```
int hashCode()
```

Returns
the hash code value for this map entry. The hash code of a map entry e is defined to be:

```
(e.getKey()==null   ? 0 : e.getKey().hashCode()) ^
(e.getValue()==null ? 0 : e.getValue().hashCode())
```

This ensures that e1.equals(e2) implies that e1.hashCode()==e2.hashCode() for any two Entries e1 and e2, as required by the general contract of Object.hashCode.

**Overrides:**

hashCode in class Object

**Returns:**

the hash code value for this map entry

**See Also:**

Object.hashCode(), Object.equals(Object), equals(Object)

### comparingByKey

```
static <K extends Comparable<? super K>,V> Comparator<Map.Entry<K,V>> comparingBy
```

Returns a comparator that compares Map.Entry in natural order on key.

The returned comparator is serializable and throws NullPointerException when comparing an entry with a null key.

**Type Parameters:**

K - the Comparable type of then map keys

V - the type of the map values

**Returns:**

a comparator that compares Map.Entry in natural order on key.

**Since:**

1.8

**See Also:**

Comparable

### comparingByValue

static <K,V extends Comparable<? super V>> Comparator<Map.Entry<K,V>> comparingBy

Returns a comparator that compares Map.Entry in natural order on value.

The returned comparator is serializable and throws NullPointerException when comparing an entry with null values.

**Type Parameters:**

K - the type of the map keys

V - the Comparable type of the map values

**Returns:**

a comparator that compares Map.Entry in natural order on value.

**Since:**

1.8

**See Also:**

Comparable

### comparingByKey

static <K,V> Comparator<Map.Entry<K,V>> comparingByKey(Comparator<? super K> cmp)

Returns a comparator that compares Map.Entry by key using the given Comparator.

The returned comparator is serializable if the specified comparator is also serializable.

**Type Parameters:**

K - the type of the map keys

V - the type of the map values

**Parameters:**

cmp - the key Comparator

**Returns:**

a comparator that compares Map.Entry by the key.

**Since:**

1.8

### comparingByValue

```
static <K,V> Comparator<Map.Entry<K,V>> comparingByValue(Comparator<? super V> cm
```

Returns a comparator that compares `Map.Entry` by value using the given `Comparator`.

The returned comparator is serializable if the specified comparator is also serializable.

**Type Parameters:**

`K - the type of the map keys`

`V - the type of the map values`

**Parameters:**

`cmp - the value Comparator`

**Returns:**

`a comparator that compares Map.Entry by the value.`

**Since:**

1.8



Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.